Московский международный колледж цифровых технологий «Академия ТОР»

Направление **«Внедрение и поддержка компьютерных систем»**

Разработка MVP программного модуля

Отчет по выполнению Зачётной работы

Выполнила:

Лясковская В.С

Новомосковск 2025 г.

Цель работы:

Спроектировать, реализовать и протестировать базовый функционал выбранной программной системы, пройдя основные этапы жизненного цикла разработки ПО.

Ход работы:

1. Концепция продукта

Проблема

Многие люди испытывают трудности с организацией своих повседневных задач, что приводит к снижению продуктивности и забыванию важных дел. Существующие решения часто перегружены функциями или требуют регистрации.

Решение

Управление задачами - простое веб-приложение для управления задачами, работающее локально в браузере без необходимости регистрации. Система позволяет быстро создавать, организовывать и отслеживать задачи с минималистичным интерфейсом.

Целевая аудитория

- Студенты, планирующие учебные задания
- Фрилансеры, управляющие проектами
- Офисные работники, организующие рабочие задачи
- Любые пользователи, нуждающиеся в простом инструменте для управления задачами

2. Функциональные требования

User-1: создание задачи

Как пользователь

Я хочу создавать новые задачи с названием, описанием и приоритетом

Чтобы фиксировать дела, которые нужно выполнить

Критерии:

- Форма содержит поля: название (обязательное), описание, приоритет
- После создания задача появляется в списке
- Задача сохраняется в LocalStorage

User-2: просмотр списка задач

Как пользователь

Я хочу видеть все свои задачи в одном списке

Чтобы иметь полное представление о предстоящих делах

Критерии:

- Отображаются все созданные задачи

- Для каждой задачи видны: название, описание, статус, приоритет
- Задачи загружаются при открытии приложения

User-3: изменение статуса задачи

Как пользователь

Я хочу изменять статус задачи (новая, в работе, завершена)

Чтобы отслеживать прогресс выполнения

Критерии:

- Доступны три статуса: "Новая", "В работе", "Завершена"
- Статус меняется кликом по кнопке
- Изменения сохраняются автоматически

User-4: редактирование задачи

Как пользователь

Я хочу редактировать существующие задачи

Чтобы актуализировать информацию

Критерии:

- Можно изменить название, описание и приоритет
- Изменения сохраняются в LocalStorage
- Интерфейс редактирования интуитивно понятен

User-5: удаление задачи

Как пользователь

Я хочу удалять ненужные задачи

Чтобы поддерживать список актуальным

Критерии:

- Кнопка удаления доступна для каждой задачи
- Запрашивается подтверждение перед удалением
- Задача удаляется из LocalStorage

3. Нефункциональные требования

Производительность

- Время отклика на действия пользователя не должно превышать 100 мс
- Приложение должно корректно работать со списком до 1000 задач
- Загрузка приложения должна занимать не более 2 секунд

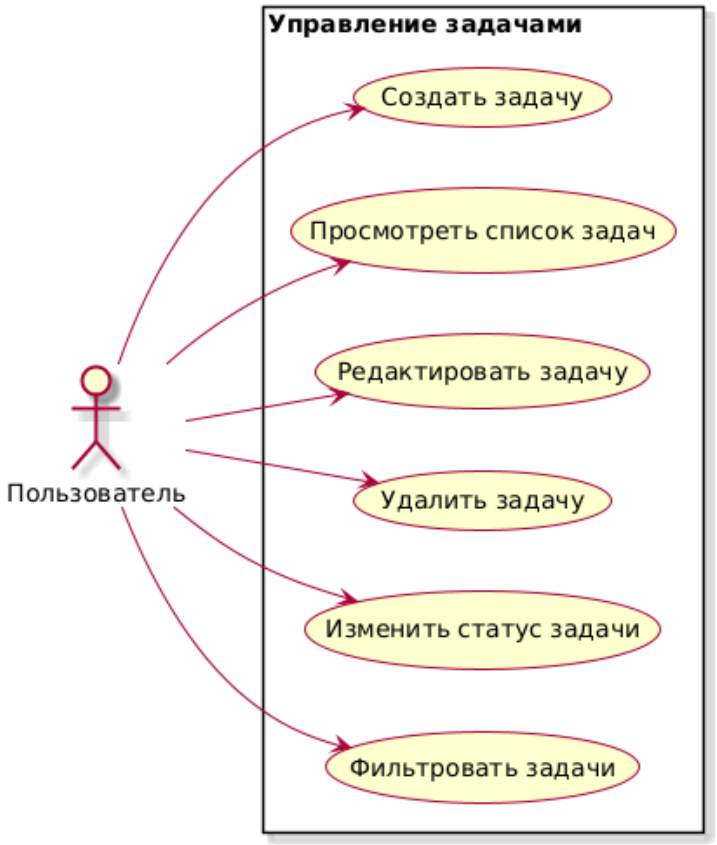
Совместимость и доступность

- Приложение должно работать в современных браузерах
- Интерфейс должен быть адаптивным и корректно отображаться на экранах от 320px
- Приложение должно работать без подключения к интернету после первой загрузки

Удобство использования

- Интерфейс должен быть интуитивно понятным без необходимости обучения
- Все интерактивные элементы должны иметь визуальную обратную связь
- Критические действия (удаление) должны требовать подтверждения

4. UML-моделирование



UC-1: создать задачу

Описание: пользователь создает новую задачу с указанием названия, описания и приоритета

Предусловия: приложение открыто

Основной поток:

1. Пользователь нажимает кнопку "Добавить задачу"

2. Система открывает форму создания задачи

3. Пользователь заполняет поля формы

4. Пользователь нажимает "Сохранить"

5. Система сохраняет задачу и отображает её в списке

UC-2: просмотреть список задач

Описание: пользователь просматривает все созданные задачи

Предусловия: приложение открыто

Основной поток:

1. Система автоматически загружает и отображает список задач

2. Пользователь видит карточки задач с информацией

UC-3: редактировать задачу

Описание: пользователь изменяет данные существующей задачи

Предусловия: в системе есть хотя бы одна задача

Основной поток:

1. Пользователь нажимает кнопку "Изменить" на карточке задачи

2. Система открывает форму с текущими данными задачи

3. Пользователь изменяет нужные поля

4. Пользователь нажимает "Сохранить"

5. Система обновляет задачу

UC-4: удалить задачу

Описание: пользователь удаляет ненужную задачу

Предусловия: в системе есть хотя бы одна задача

Основной поток:

1. Пользователь нажимает кнопку "Удалить" на карточке задачи

2. Система запрашивает подтверждение

3. Пользователь подтверждает удаление

4. Система удаляет задачу из списка

UC-5: изменить статус задачи

Описание: пользователь меняет статус задачи (новая - в работе - завершена)

Предусловия: в системе есть хотя бы одна задача

Основной поток:

1. Пользователь нажимает кнопку изменения статуса

2. Система переводит задачу в следующий статус

3. Система обновляет отображение задачи

UC-6: фильтровать задачи

Описание: вользователь фильтрует задачи по статусу

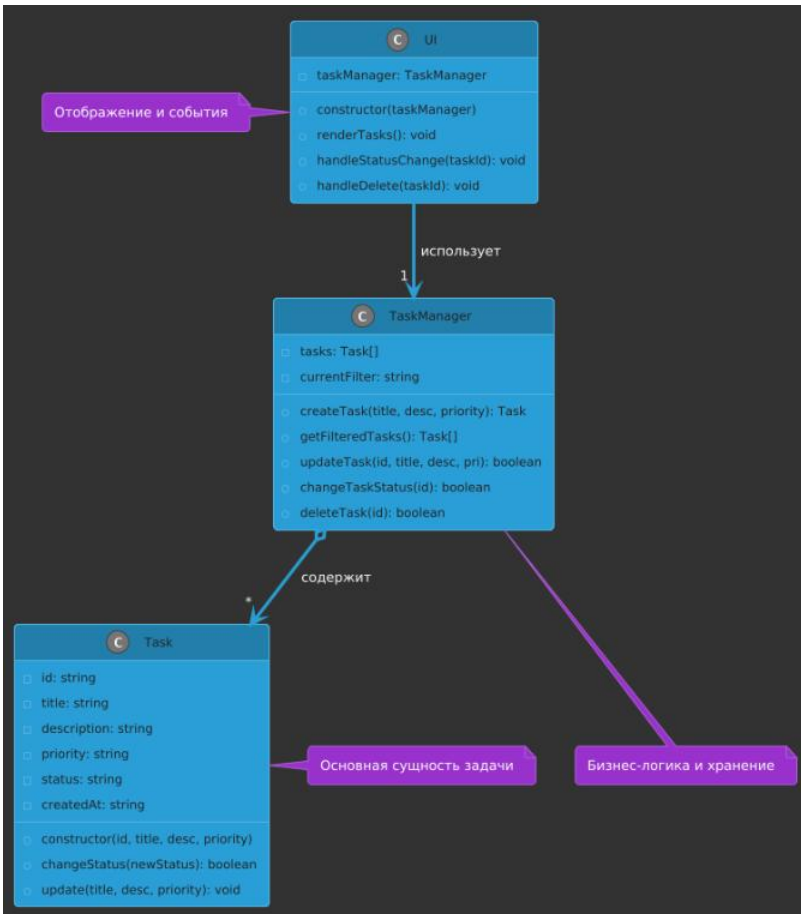
Предусловия: приложение открыто

Основной поток:

1. Пользователь выбирает фильтр (все, новые, в работе, завершенные)

2. Система отображает только задачи с выбранным статусом

UML диаграмма классов



Класс Task

Назначение: Представляет отдельную задачу в системе

Атрибуты:

- id - уникальный идентификатор задачи

- title - название задачи
- description - описание задачи
- priority - приоритет (low, medium, high)
- status - статус (new, in-progress, completed)
- createdAt - дата и время создания

Методы:
- changeStatus() - изменяет статус задачи
- update() - обновляет данные задачи
- getNextStatus() - возвращает следующий статус в цикле
- getStatusButtonText() - возвращает текст для кнопки статуса
- getStatusText() - возвращает читаемое название статуса
- getPriorityText() - возвращает читаемое название приоритета

Класс TaskManager
Назначение: Управляет коллекцией задач и бизнес-логикой

Атрибуты:
- tasks - массив всех задач
- currentFilter - текущий активный фильтр

Методы:
- createTask() - создает новую задачу
- getTask() - получает задачу по ID
- getAllTasks() - возвращает все задачи
- getFilteredTasks() - возвращает отфильтрованные задачи
- updateTask() - обновляет существующую задачу
- changeTaskStatus() - изменяет статус задачи
- deleteTask() - удаляет задачу
- setFilter() - устанавливает фильтр
- saveToStorage() - сохраняет данные в LocalStorage
- loadFromStorage() - загружает данные из LocalStorage
- getStatistics() - возвращает статистику по задачам

Класс UI

Назначение: Управляет пользовательским интерфейсом и взаимодействием

Атрибуты:
- taskManager - ссылка на менеджер задач
- modal - модальное окно
- taskForm - форма создания/редактирования
- taskList - контейнер списка задач
- editingTaskId - ID редактируемой задачи
- addTaskBtn - кнопка добавления задачи
- filterBtns - кнопки фильтрации

Методы:
- initializeElements() - инициализирует DOM элементы
- showAddTaskModal() - показывает модальное окно для создания
- showEditTaskModal() - показывает модальное окно для редактирования
- closeModal() - закрывает модальное окно
- renderTasks() - отрисовывает список задач
- createTaskCard() - создает HTML карточку задачи
- handleStatusChange() - обрабатывает изменение статуса
- handleDelete() - обрабатывает удаление задачи
- handleFormSubmit() - обрабатывает отправку формы
- handleFilter() - обрабатывает фильтрацию
- escapeHtml() - экранирует HTML для безопасности
- initializeEventListeners() - инициализирует обработчики событий

Связи между классами

1. TaskManager содержит Task (композиция 1:)
   - TaskManager управляет коллекцией объектов Task
   - При удалении TaskManager удаляются все Task

2. UI использует TaskManager (ассоциация 1:1)
   - UI делегирует бизнес-логику TaskManager
   - UI отображает данные из TaskManager

3. UI создает Task через TaskManager (зависимость)

- UI не создает Task напрямую
- Все операции с Task идут через TaskManager

5. Тест-кейсы

ID	Название тест-кейса	Приоритет	Статус выполнения	Фактический результат	Баг-ID
TC-001	Создание с обязательными полями	Высокий	✅	Создана, статус «Новая», приоритет «Средний»	—
TC-002	Создание со всеми полями	Высокий	✅	Все поля (название, описание, приоритет) сохранены	—
TC-003	Создание без названия	Высокий	✅	Ошибка: «Пожалуйста, введите название задачи»	—
TC-004	Отмена создания	Средний	✅	Окно закрыто, задача не создана	—
TC-005	Редактирование всех полей	Высокий	❌	Приоритет не изменился (остался «Средний»)	Баг-1: приоритет не сохраняется
TC-006	Отмена редактирования	Средний	✅	Изменения отменены, данные остались прежними	—
TC-007	Статус Новая → В работе	Высокий	✅	Статус → «В работе», кнопка → «Завершить»	—
TC-008	Статус В работе → Завершена	Высокий	✅	Статус → «Завершена», кнопка → «Возобновить»	—

Выводы:

В курсовой работе спроектировано, реализовано и протестировано простое локальное веб-приложение для управления задачами

Решена проблема отсутствия лёгкого оффлайн-планировщика: приложение позволяет создавать, просматривать, редактировать, удалять задачи, менять статус и фильтровать список

Разработана архитектура на трёх классах: Task, TaskManager (логика + LocalStorage), UI (интерфейс). Построены UML-диаграммы прецедентов и классов

Обеспечены быстрый отклик, адаптивность, интуитивность и безопасность базового уровня. Приложение успешно протестировано

Получен рабочий MVP для повседневного использования. Работа охватывает полный цикл разработки: анализ - проектирование - реализация - тестирование.